Vulcan, project specification (Track 2, Private AI Agents)

> Submission requirement: project specification document with scenarios, architecture, capabilities and optimization details.

1. Scenario and users

Developers working on private or regulated codebases cannot send source code to cloud LLM APIs. Vulcan is a developer-productivity agent that runs entirely on a local AMD Radeon GPU: codebase Q&A with citations, bug localization, test execution and code edits, with zero data egress.

2. Architecture

```
user ── CLI (typer) ── Agent (ReAct loop, JSON tool protocol)
                        ├─ RAG index    SQLite + cosine, embeddings (2.1)
                        ├─ Tools        search_code, read_file, grep,
                        │               run_cmd*, write_file
                        ├─ Memory       durable per-project notes
                        └─ LLM client ── OpenAI-compatible endpoint
                                          └─ vLLM on ROCm / Radeon GPU

* run_cmd is allowlisted; all file access is path-jailed to the indexed root.
```

2.1 Where each half actually runs

Stated precisely, because the privacy claim depends on it and it is easy to overstate.

`vllm serve Qwen/Qwen3-8B` starts with task=generate. The resulting server advertises `/v1/chat/completions`, `/v1/completions`, `/v1/responses`, `/v1/messages`, `/tokenize` and `/v1/models`, and **no `/v1/embeddings`**; that route returns 404. Confirmed against the live instance rather than assumed.

Serving embeddings from vLLM requires a second process started with `--task embed` against an embedding model, and Radeon Cloud permits one active instance per account. So in the configuration measured here:

stage	runs on
agent reasoning and generation	Radeon GPU, vLLM on ROCm
RAG embeddings at index time	local embedding model
retrieval, chunking, tool execution	local CPU

Both endpoints are ones the operator controls, so the no-egress property holds: no source code reaches a third party. But only generation is GPU-served in this setup, and the project does not claim otherwise.

Pointing `VULCAN_EMBED_MODEL` at an `--task embed` vLLM instance moves embeddings onto the GPU too, at the cost of a second instance.

2.2 Model choice and deployment plan

Generation model: `Qwen/Qwen3-8B`. Chosen for three reasons specific to this workload. It is small enough to serve from a single consumer Radeon at bf16 (7.67 GiB of weights, leaving room for KV cache at `--gpu-memory-utilization 0.92`), its 40960-token context comfortably holds retrieved code chunks plus a ReAct scratchpad, and it follows the strict JSON tool protocol in `vulcan/agent.py` reliably enough to drive the loop without native tool-calling support.

Qwen3 reasons by default, which is the wrong trade here and is disabled per request; see section 4.3.

Embedding model: `mxbai-embed-large`, served locally. Section 2.1 explains why it cannot share the Radeon instance.

Deploying on the Radeon (ROCm). Image `vllm-dev:rocm7.2.1_navi_ubuntu22.04_py3.10_pytorch_2.9_vllm_0.16.0`, one GPU:

```
vllm serve Qwen/Qwen3-8B --host 0.0.0.0 --port 8000 --gpu-memory-utilization 0.92
```

Cold start is roughly four minutes: weights load in about 55s, then `torch.compile` builds and caches the graphs. Subsequent starts reuse that cache and load weights in about 5s. Point the agent at it:

```
export VULCAN_BASE_URL=https://<host>/spaces/<instance-id>/8000/v1
export VULCAN_API_KEY=<per-instance key>
export VULCAN_MODEL=Qwen/Qwen3-8B
export VULCAN_ENABLE_THINKING=false
export VULCAN_EMBED_BASE_URL=http://localhost:11434/v1
```

No code change is required: `vulcan/llm.py` speaks OpenAI-compatible chat against any `base_url`. The full operational runbook, including the launch rate limits and boot failure modes actually hit during development, is in `radeon-deploy.md`.

Deploying locally (no GPU). Ollama serves both halves from one endpoint, which is the development configuration and the laptop baseline in section 4:

```
ollama pull qwen3:4b-instruct && ollama pull mxbai-embed-large
export VULCAN_BASE_URL=http://localhost:11434/v1
export VULCAN_MODEL=qwen3:4b-instruct
```

3. Capabilities

- Multi-turn codebase Q&A with file:line citations, carrying conversation context between turns so a follow-up like "and which one stores memory?" resolves against what was just said
- Autonomous tool use: semantic search, file reading, grep, test runs, edits
- Hybrid retrieval: dense cosine plus literal term overlap
- Persistent memory across sessions per project

- Backend-agnostic: same agent on Ollama, llama.cpp or vLLM-ROCm
- Privacy that can be checked rather than believed: `vulcan privacy-check`

3.0 The privacy claim is testable

This track is about private agents, and the claim that no token reaches a third party is the whole proposition. Stated in a README it is not evidence: the code looks local, the document says local, and a reader has to take both on trust.

`vulcan privacy-check` runs a real task with every outbound request recorded, and reports each host against the endpoints actually configured:

```
Outbound traffic during one agent run
host       allowed
localhost  yes
4 request(s); allowed endpoints: localhost
No traffic left the configured endpoints.
```

It hooks `httpx.Client.send`, the one choke point all of this project's traffic passes through, so a call site added later is covered without anyone remembering to update a list. Two limits are stated rather than glossed: it sees Vulcan's own HTTP, and `run_cmd` executes the user's commands unsandboxed, so a test suite the agent runs could reach the network on its own account. And a local endpoint is trusted because it is the one you configured; point `VULCAN_BASE_URL` at a hosted API and the check will faithfully name that host, which is the point of it.

A check that cannot fail proves nothing, so the test suite makes it fail: a deliberate request to `api.openai.com` inside the recorder must be reported as unexpected.

3.1 Retrieval is hybrid, and the weight was swept

Dense cosine alone ranked `rag.py` fourth for "which module implements the semantic index", behind `cli.py` and `agent.py`, with every score bunched between 0.591 and 0.648. A 60-line chunk is dominated by its file header, and every header resembles every question. The agent then answered, correctly given what it was shown, that the index could not be identified.

Adding literal term overlap (a match in the path counts triple) fixes the ranking. The weight was swept rather than chosen, over seven questions whose correct file is not in dispute:

lexical weight	top-1	top-3
0.0, dense only	5/7	7/7
0.15, shipped	**6/7**	**7/7**
0.25	5/7	7/7
0.5	4/7	6/7
1.0	4/7	6/7

`vulcan rag-bench` reproduces the table.

This table replaced an earlier one, and the correction is the interesting part. The first sweep found a plateau: 0.15, 0.25 and 0.5 all scored 6/7 and 7/7, so 0.25 shipped as the middle of a stable range rather than the edge of one. That sweep ran against an index which turned out to be indexing its own benchmark output. `.json` was in the source extension list, so fifteen result files and a pytest cache were embedded as code, and a query about recording outbound hosts returned two JSON files and a shell script while `vulcan/privacy.py` never appeared at all.

With the corpus restricted to source, the plateau disappears. Dense-only improves, because the noise it was ranking is gone, and every weight above 0.15 falls away, 0.5 and 1.0 losing a top-3 hit as well. The lexical weight had been paying for the noise, and once the noise went, carrying it stopped being free.

Seven queries decide this, so a single query separates 0.0, 0.15 and 0.25, and the shipped value is not claimed to be optimal to two decimal places. What the sweep supports firmly is the shape: weight the lexical signal lightly, and never heavily.

3.2 Task decomposition was measured and rejected

An explicit plan step was added to the protocol, so a multi-part request would be decomposed before any tool ran. It does not pay, on either model tested:

model	plan off	plan on	ever planned	extra wall clock
qwen3:4b-instruct	3/4	3/4	no	+18.2 s
qwen3-ws:32k	4/4	4/4	no	+20.9 s

Neither model emitted a plan even once, so completion was identical while the instructions occupied the system prompt on every call. It ships disabled; `VULCAN_PLAN=1` restores it and `vulcan plan-bench` reproduces the result, the same treatment given to AWQ in section 4.

What did improve completeness was fixing retrieval: the larger model goes from 3/4 to 4/4 on the same questions once the right file is actually returned.

3.3 Three defects the measurements exposed

None of these were visible until something was measured end to end:

- **A search observation was unbounded.** Every other tool clips its output; `search_code` returned six full 60-line chunks, enough to exceed a 4096-token context on its own. It surfaced as an opaque 400 mid-session.
- **The failure was unreadable.** `raise_for_status()` on a streaming response reports the status and nothing else, and reading the body afterwards raises `ResponseNotRead`, so the server's explanation was unreachable exactly when it was needed. The body is now read first and included in the error, which is how the context overflow above was identified.
- **A reply of `{"thought": ...}` alone was dispatched as a tool named `""`**, answered `ERROR: unknown tool`, and after a few rounds the agent told the user "I cannot proceed because all tools are unavailable." It now asks for a corrected reply.

4. ROCm optimization

4.1 Measurement methodology

Every number below comes from `vulcan bench`, which streams three fixed prompts (short, medium, long) and records time-to-first-token and generation rate. Repeat counts differ by run and are recorded in each JSON file: the Radeon decode set is `repeats: 5`, both prefill sets are 3, the laptop decode baseline is 2, and the concurrency sweeps take one sample per level. The harness is the same code on both platforms, so the only variable is the backend.

The hardware. Captured on the instance itself and committed verbatim to `bench-results/radeon-device.txt`:

```
gcnArchName:             gfx1100        # RDNA 3, Navi 31
total_memory_GiB:        48.0
multi_processor_count:   48
torch:                   2.10.0+rocm7.2.4.git3d3aa833
hip:                     7.2.53211
Card Model:              0x744b
```

`rocm-smi` cannot reach libdrm inside the container, so it reports the PCI model id rather than a marketing name. The torch device properties are what the ROCm driver actually reports.

Two honesty caveats that shape how the numbers should be read:

- **The Radeon runs are measured over an HTTPS proxy, not on the box.** The instance's SSH port is not reachable from the client network, so the client talks to vLLM through the Radeon Cloud spaces proxy. Median round-trip to that proxy was **0.31s** during the benchmark run (`proxy_rtt_s_median` in `bench-results/radeon-vllm-qwen3-8b-nothink.json`).

  That is larger than the short and medium Radeon TTFTs reported below (0.306s and 0.292s), so **the single-stream TTFT column is inside proxy noise and should not be read as a GPU measurement at all.** Only the long-prompt TTFT (0.422s) sits clear of it. The decode-rate and concurrency numbers are not affected the same way, because they measure sustained streaming over many seconds rather than a single round trip.

- **"Tokens/sec" is really content-deltas/sec.** The harness counts streamed SSE deltas, not tokenizer tokens. Deltas can coalesce in transit, so the proxied Radeon figure is a slight undercount relative to the local run.

The link to a remote GPU is not reliable, and the agent had to be hardened for it rather than assumed away. During a recorded demo run the proxy dropped mid-request with `ConnectError: [SSL: UNEXPECTED_EOF_WHILE_READING]`, killing an agent turn outright while the identical endpoint served a benchmark seconds later. `LLM.chat` now retries transport failures three times with linear backoff, and a stream is only retried while nothing has been emitted, so a half-delivered answer is returned truncated rather than silently duplicated.

Cold start matters and is reported rather than hidden: the first two repetitions of each prompt carry cache-warming cost, and the run is only stable from roughly the third repetition. Taking the median of 5 keeps a single cold outlier from dominating while still not discarding it.

4.2 Optimization axes

Single-stream tokens/sec is the number everyone reports and it is the least interesting one for an agent. A ReAct loop issues many short steps, often several at once, over a context stuffed with retrieved code. So Vulcan measures three axes, each a separate reproducible command:

axis	command	why it matters for an agent
decode	`vulcan bench`	raw generation speed, the usual headline
concurrency	`vulcan bench-concurrency`	agent fleets issue parallel steps; does the backend batch or collapse?
prefill	`vulcan bench-prefill`	RAG stuffs context, so TTFT against input length is the latency users feel

Concurrency is where the hardware argument actually lives

Measured on the laptop (Ollama, `qwen3:4b-instruct`):

concurrent requests	aggregate	per request	median TTFT
1	28.4 chunks/s	28.4	4.36s
2	20.0 chunks/s	10.0	42.21s

Adding a second concurrent request made **aggregate** throughput *fall* and pushed TTFT out by nearly 10x. Ollama serves these without continuous batching, so the second request does not share the GPU, it queues behind the first. Higher levels were not run because the curve was already inverted.

That is the honest case for the Radeon: not "the GPU is faster at one stream", but "vLLM's continuous batching keeps the curve going the right way while the laptop's inverts". Section 4.3 has the matching Radeon curve.

A note on model parity

The Radeon runs `Qwen/Qwen3-8B` and the laptop runs `qwen3:4b-instruct`. These are not the same weights, so absolute decode numbers are not a clean hardware isolation and are not presented as one. The concurrency *shape*, whether aggregate throughput rises or falls as load increases, is a property of the serving stack rather than the parameter count, and the Radeon is carrying the larger model while doing it.

An apples-to-apples run was attempted by re-serving `Qwen/Qwen3-4B-Instruct-2507` on the Radeon to match the laptop weights. It loaded (7.67 GiB) and then died twice during `torch.compile`, with the instance reaped and logs cleared, during an announced platform maintenance window. Rather than report a number that could not be reproduced, the mismatch is disclosed here.

4.3 Results

The decode, prefill and concurrency tables regenerate from committed JSON with `vulcan bench-compare`; the exact command is given above each one. The thinking-mode table in 4.4 does **not**: it was

measured live during the recorded session and the harness does not persist the delta-count and wall-clock columns it reports (`vulcan/bench.py` stores `ttft_s_median` , `chunks_per_s_median` and `repeats` only). It is kept because the effect is large and reproducible by rerunning the two commands shown, but it is not evidence in the same sense as the others.

Decode, single stream

`vulcan bench-compare bench-results/ollama-m4air-qwen3-4b.json bench-results/radeon-vllm-qwen3-8b-nothink.json`

case	laptop 4B ttft (s)	radeon 8B ttft (s)	laptop chunks/s	radeon chunks/s	speedup
short	0.2	0.306	32.3	24.2	0.75x
medium	0.225	0.292	20.2	24.0	1.19x
long	0.853	0.422	15.2	23.9	1.57x

The Radeon loses the short prompt and wins the long one, while carrying twice the parameters. The reason is visible in the shape: laptop throughput decays as the prompt grows (32.3 to 15.2) whereas the Radeon holds flat (24.2 to 23.9). Long-prompt TTFT halves, 0.853s to 0.422s, and that figure still has the proxy round-trip inside it.

Concurrency, the decisive axis

`vulcan bench-compare bench-results/ollama-m4air-concurrency.json bench-results/radeon-vllm-concurrency.json`

concurrent	laptop ttft (s)	radeon ttft (s)	laptop chunks/s	radeon chunks/s	speedup
1	4.359	1.486	28.4	23.5	0.83x
2	42.212	1.248	20.0	42.9	2.15x
4	n/a	0.553	n/a	87.0	n/a
8	n/a	0.411	n/a	179.2	n/a

Read the two curves rather than any single cell:

- **Radeon**: 23.5 to 179.2 chunks/s across an 8x load increase, a **7.63x scaling factor, 95% efficiency**. Per-request rate barely moves (23.5 to 22.4). Median TTFT *improves* 3.6x. Wall clock for the whole batch stays between 51s and 65s whether it is serving 1 request or 8, against an 8x increase in work. The on-camera run spans 49s to 63s.
- **Laptop**: adding a single extra request *reduces* aggregate throughput to 0.70x and pushes TTFT from 4.36s to 42.21s. Levels above 2 were not run because the curve had already inverted.

At two concurrent requests the Radeon answers in 1.2s where the laptop takes 42.2s, a **34x** difference in the latency a user actually waits. This is the real argument for the GPU, and it is invisible to any single-stream benchmark.

Prefill, the context budget

input words	radeon ttft (s)
128	0.804 (cold start)
512	0.316
2048	0.399
8192	2.091

Context is close to free up to ~2048 words, then 4x more context costs 5x the TTFT. That is a direct instruction to the RAG layer: keep retrieved context under about 2000 words and TTFT stays under half a second.

Thinking mode, a serving-config win

mode	deltas generated	wall clock	delta rate
thinking (Qwen3 default)	4058	170.9s	23.8/s
`enable_thinking: false`	1168	49.0s	24.0/s

Identical generation rate. Thinking emits ~3.5x more tokens for the same task, so per-step latency falls by the same factor. Reproducible from the CLI with `VULCAN_ENABLE_THINKING=false`.

Serving config: what actually produces the scaling

The concurrency result above is on vLLM's stock settings, which invites a fair question: is the default right, or was throughput left on the table? The claim being made is that continuous batching produces the scaling, so the way to test it is to take batching away and see whether the scaling goes with it.

`--max-num-seqs 4` caps the batch at four sequences. Everything else is identical: same model, same instance type, same harness, same prompts.

```
vulcan bench-compare bench-results/gmu092-stock-conc.json bench-results/maxseqs4-conc.json
```

concurrent	stock agg (chunks/s)	capped at 4	stock TTFT	capped TTFT
1	20.0	21.6	1.679s	5.267s
4	78.1	86.7	7.406s	1.509s
8	**162.5**	**88.0**	1.629s	**25.489s**

Scaling from 1 to 8: 8.12x stock, 4.07x capped at 4.

The capped run scales to 4.07x, which is its cap almost exactly. That is the control the earlier claim needed: the scaling is continuous batching, not bandwidth or clock, and `--max-num-seqs` is the knob that governs it.

Three things follow, and the second is the one that matters for the agent:

- At or below the cap the capped config is marginally *faster* (86.7 against 78.1 at four concurrent). Restricting the scheduler is not free but it is not a loss either while the load fits.
- Past the cap it collapses. Aggregate throughput falls 46%, per-request rate halves (20.3 to 11.0 chunks/s), and **median TTFT goes from 1.6s to 25.5s** because half the requests wait for a slot. A 25-second first token is not a slow agent, it is a broken one.
- So the stock configuration is already correct for this workload, and this is the measurement that says so rather than an assumption. Tuning effort belongs elsewhere.

Single-stream decode is unchanged between the two configs (24.3 / 24.0 / 23.9 chunks/s stock against 24.1 / 23.9 / 23.8 capped, `repeats: 5` each), which is the isolation check: one request never reaches a batch cap of four, so the flag provably touched only the batching path and nothing about raw generation.

TTFT at a single sample per level is noisy: the 7.406s stock figure at four concurrent is an outlier against 1.6s either side of it, and the 5.267s capped figure at one concurrent is the same artefact. Aggregate throughput is the robust signal here and it is what the conclusion rests on. Both files carry `repeats: 1` per level.

Quantization: it costs throughput on this card

AWQ 4-bit was the obvious next thing to try, so it was tried. `Qwen/Qwen3-8B-AWQ` served on the same instance, same flags, same harness.

First, it works at all: AWQ loads and serves on **gfx1100 under ROCm vLLM 0.16**, which is not a given, since much of the quantized-kernel tooling assumes CUDA. Cold start was about six minutes including the weight download.

```
vulcan bench-compare bench-results/gmu092-stock-conc.json bench-results/awq-conc.json
```

concurrent	fp16	AWQ 4-bit	AWQ / fp16
1	20.0 chunks/s	14.4	0.72x
4	78.1	53.3	0.68x
8	162.5	111.1	0.68x

Single-stream decode agrees: 23.9 to 24.3 chunks/s at fp16 against 15.5 to 15.7 at AWQ, a consistent **0.65x**.

Quantization is a loss here, and the reason is the card. AWQ buys memory by paying compute: weights are unpacked on every forward pass. That trade is worth it when the model does not otherwise fit, or when memory bandwidth is the binding constraint. On a 48 GB card serving an 8B model, neither is true. The weights already fit with room for a large KV cache, so the dequantization cost is paid for a benefit that is not needed, and throughput drops by a third.

Batching still behaves: AWQ scales 7.7x from 1 to 8 concurrent against fp16's 8.12x, so the continuous-batching result in the previous section is a property of the serving stack rather than of the weight format.

The honest conclusion for this agent is to stay at fp16. Quantization becomes the right call on a smaller card, or for a model in the 30B-plus range where fp16 stops fitting in 48 GB, and this measurement is what says which regime you are in.

4.4 Quantization: rejected on one model, required on another

Section 4.3 rejected AWQ. On Qwen3-8B it returned 0.68x of fp16 throughput at every concurrency level, because a 48 GB card serving an 8B model has no memory problem to buy its way out of. That conclusion stands, and it is only half the picture.

Quantization on this card is a capacity decision, not a speed one. The test is a model that fp16 cannot hold at all:

	fp16	**Q4_K_M**
Qwen3-32B weights	61.1 GB (32.8B params x 2 bytes)	**18.4 GB**
Fits in 49,136 MB of VRAM	**no**	yes
Measured VRAM in use	n/a	27,002 MB

Loaded and served on the Radeon, with every layer on the GPU rather than spilled to host memory:

```
llama_kv_cache: layer  0..63: dev = ROCm0        all 64 layers
ROCm0 model buffer size   = 18423.65 MiB
ROCm0 KV buffer size      =  4096.00 MiB         (n_ctx 16384)
ROCm0 compute buffer size =  2136.01 MiB
```

VRAM goes from 26 MB idle to 27,002 MB loaded. The numbers above are captured from the running server into `bench-results/quantization.json` rather than typed in.

Serving. llama-cpp-python 0.3.34 built from source against ROCm 7.2.4 with `-DGGML_HIP=ON -DAMDGPU_TARGETS=gfx1100`. The resulting `libggml-hip.so` links `libamdhip64`, `librocblas` and `libhipblas`, which is what distinguishes a real HIP build from a CPU one that merely imports. It exposes an OpenAI-compatible API on port 8000, which the Radeon Cloud spaces proxy publishes, so Vulcan reaches it by changing one environment variable and no code.

Throughput, `bench-results/radeon-llamacpp-qwen3-32b-q4.json`:

prompt	**TTFT**	**generation**
short	1.287 s	26.8 chunks/s
medium	0.369 s	21.2 chunks/s
long	1.772 s	23.3 chunks/s

These are single-stream figures from a different server on a different model size, so they are **not** comparable to the vLLM concurrency table in 4.3 and are not presented as an improvement on it. The claim here is narrower and stronger: a model that does not fit at all in fp16 runs, on the GPU, through the cloud API, and answers a real agent task correctly in 53 seconds.

A trap worth writing down. The first attempt served the model with `--chat_format chatml`. Qwen3's GGUF ships its own template, which also honours `enable_thinking`, and forcing chatml over it stripped the tool protocol out of the system prompt. The agent then made no tool calls at all and answered from prior belief, citing `vulcan/search/index.py` and `vulcan/memory/persistence.py`, neither of which exists. Dropping the flag so the GGUF template is used produced `search_code` followed by the correct answer, `vulcan/rag.py` and `vulcan/memory.py`. The failure looked like a weak quantized model and was actually a wrong flag, which is a good reason to check citations against the filesystem rather than trusting fluent output.

4.5 What was not measured, and why

Stating the boundary rather than implying the map is complete:

- **VRAM saving was not measured directly.** AWQ's benefit is a smaller resident model, and the instance exposes only an inference endpoint through the proxy, with no shell (ssh refuses on port 31200) and no rocm-smi from the client. The throughput cost is measured; the memory saving is inferred from the format and is not claimed as a number here.
- **No `--gpu-memory-utilization` or chunked-prefill sweep.** The batch-cap experiment above answered the question those were going to be asked for, which was whether the stock serving config is leaving throughput unclaimed.
- **No GPU embedding throughput.** Serving embeddings needs a second vLLM process started with `--task embed`, and Radeon Cloud allows one active instance per account, so it was not reachable from this configuration at all. Section 4.1 explains the consequence.

5. What the demo video shows

`demo.mp4`, 3 m 42 s, recorded against the live Radeon endpoint. Nothing is staged: every command really runs and the output is whatever the machine returned.

1. Title card and the premise (7s)
2. `vulcan index` on this repository, then an agent question answered with cited files, served by vLLM on the Radeon
3. `vulcan bench` decode and prefill runs against the Radeon endpoint
4. `vulcan bench-concurrency --levels 1,4,8`, the 6.96x result, captured live
5. `vulcan bench-compare` head-to-head against the laptop baseline
6. End card with the committed numbers (9s)